Multi module programming

include → code gets put into one single file ⇒ no multi-module.

directions: extern / global } ⇒ exchange zone data (functions, variables)
(import / export)

n compile units → n modules (obj. files)

Static linking ⇒ joining multiple binary modules.

global ⇒ exporting this resource
extern ⇒ importing mechanism ; it's public

static ⇒ stop the exporting (make the var local).

Exam: NO multi module with C. (only w/ ASM)

Look at the course page for exam.

global var1, subroutine2 ⟺ extern var1, subroutine2

FILE1.ASM

FILE2.ASM

CDECL => the caller can use as many param. as he wants, but he needs to clean the stack.

STDCL => (WINDOWS) => fixed param., but the fce. cleans the stack by itself.

void asm_start(void); ≡ extern void asm_start(void)

Afjoe.c

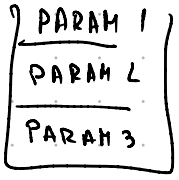
- declares asm_start (CDECL)
- defines fce. afjoe

Vector.masm

- defines asm_start
  ↓
  will use Afjoe.

CDECL

- specific to C
- variable no. of param
- cleanup by the caller
- volatile: EAX, ECX, EDX, flags
- result stored: EAX, EDX:EAX ST0(FPU)



STDCALL

- fixed no. param.
- clean by callee

Subroutine call

Call code (the caller's responsibility)

a) save volatile resources: push register, (EAX, ...)
b) ensure compliance w/ constraints (aligned ESP, DF=0)

c) Saving the returning address and performing the call

Entry code (The callee's responsibility)

a) Building a new stack frame - push EBP
mov EBP, ESP.

b) allocating space for local variables sub esp, nr_bytes

c) Saving non-volatile resources expected to be modified

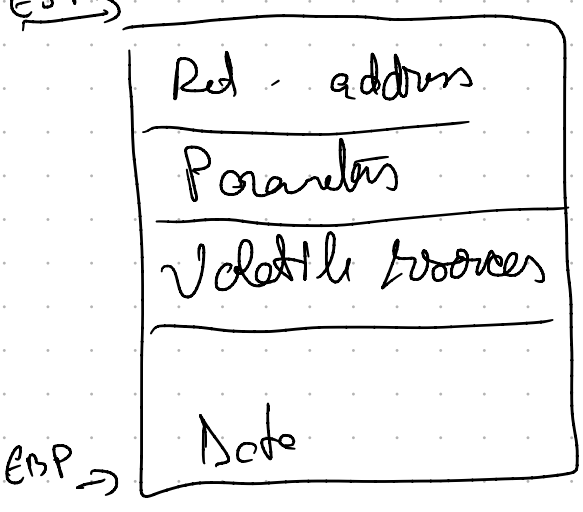
Why do we have 3 push eax
dec eax
push eax ?

push eax => for later comparison.

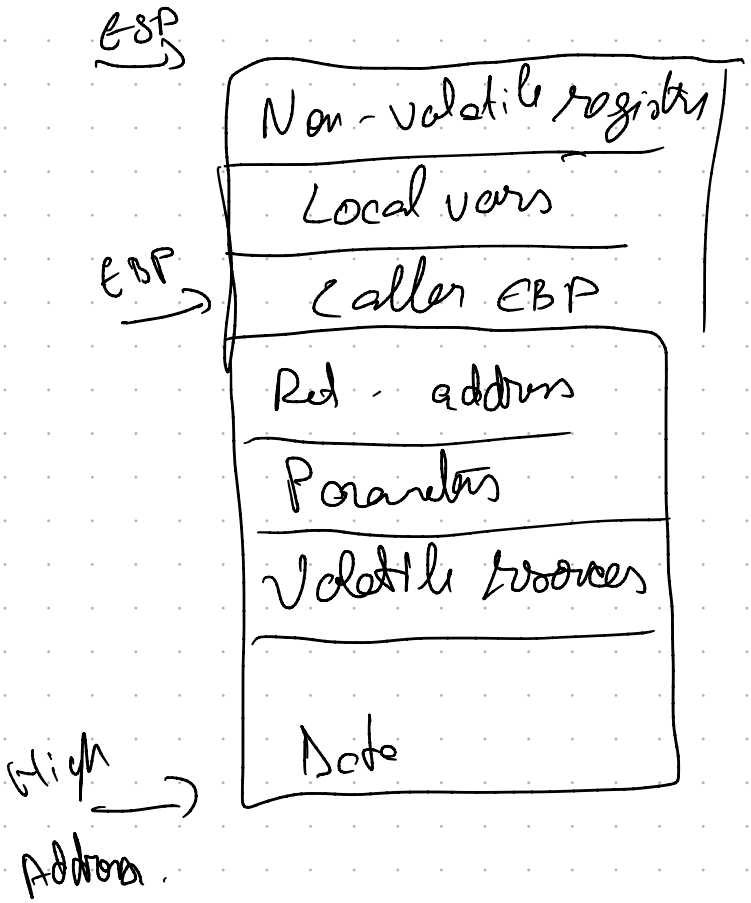
dec eax
push eax } => fuction call

the call code for the new fuc.

Because the fist call of fact(100) is automatically generated by the C copiler (not the fact...) have to be made by our code !



Before the factorial.



Exit codes

a) Restore non-volatile res.
b) release local vars.
c) deallocating the stack frame.
d) returning from the function depending on the address